Topic: Terraform Provisioners — `file`, `remote-exec`, `local-exec` & the `connection` Block

(Your notes, cleaned up from dictation + screenshots — Lecture 4)

Quick recall

This lecture picked up the two items left over from Chapter 03's list: **provisioners** and the **plugin cache**. This session covered provisioners only. Plugin cache is the next topic.

Term	One-liner
Terraform provisioner	A tool used **inside a Terraform configuration** to **execute scripts or commands** on a local or remote machine as part of the resource provisioning process
`file` provisioner	Copies a file/directory **from the Terraform machine to the new resource** (works like `scp`)
`remote-exec` provisioner	Logs in to the new resource and **runs commands inside it** (works like `ssh` + run)
`local-exec` provisioner	Runs a command **on the Terraform machine itself** (e.g. save the new server's IP locally)
`connection` block	**How** Terraform logs in for `file` / `remote-exec`: type, user, private key, host
Terraform plugin cache	Local storage that **saves provider plugins** so Terraform is faster and more reliable *(next chapter)*

> **Definition (from the lecture's reference):** Terraform provisioners are a built-in feature used to execute scripts, commands, or file transfers on local or remote machines during the **creation or destruction** of a resource. They are an **"imperative escape hatch"** in Terraform's otherwise declarative model. They fill the gap when a cloud provider has no native way to bootstrap or configure a newly deployed resource.

---

1. The idea: SCP and SSH, but done by Terraform

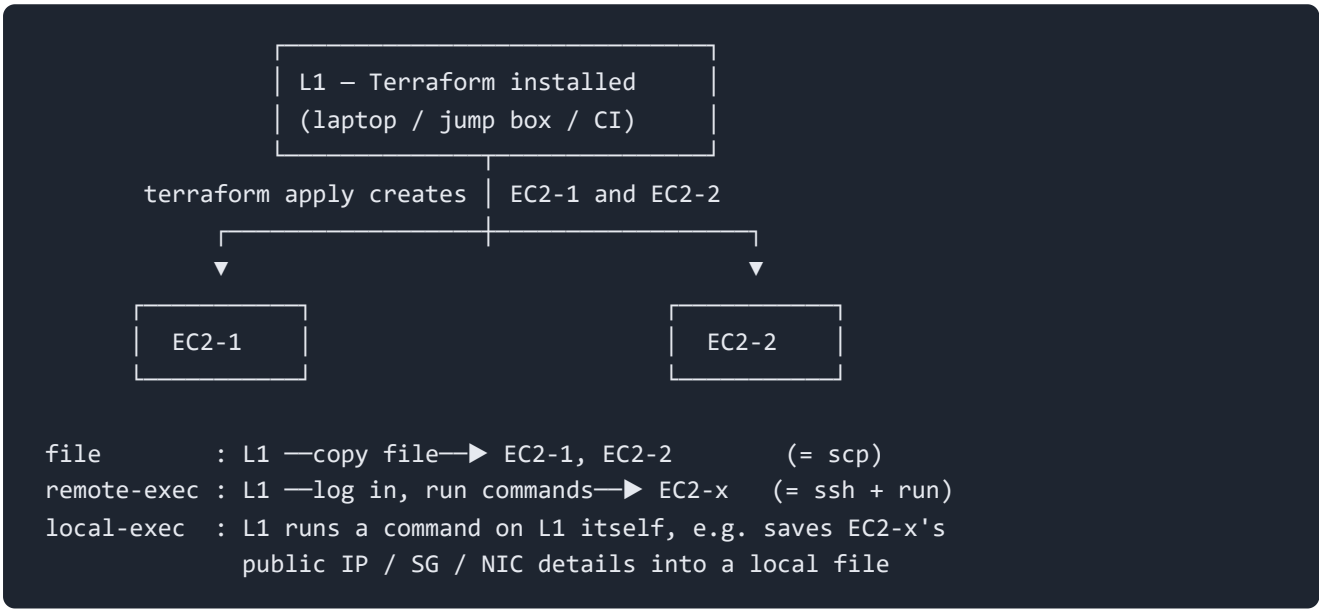
In plain Linux we already have two commands for working with other machines:

Linux command	What it does
`scp` (secure copy)	**Copy a file** from this machine to a remote host
`ssh`	**Log in** to a remote host (IP + username + password/key) and run things there

Say we have one machine **L1** and three others: **L2, L3, L4**.

- To log in from L1 to L3 → `ssh user@<L3-ip>`.
- To send one file from L1 to the other three → `scp file user@<ip>:/path` for each.

The same concept exists in Terraform, only it's called a *provisioner*.



The three provisioners, in my words

1. `file` **provisioner.** L1 has Terraform installed. From it we create two EC2 instances, EC2-1 and EC2-2, and we want to move a file from L1 onto both. The `file` provisioner copies it. **Here the file provisioner works as SCP.**
2. `remote-exec` **provisioner.** The file is copied. Now, from L1, we want to **run commands or scripts inside** EC2-1 and EC2-2 (install packages, move the config into place, restart a service). That's `remote-exec`.
3. `local-exec` **provisioner.** While L1 is provisioning EC2-1 and EC2-2, we want to **save some information/metadata about them on L1 locally**: public IP, security groups, network card details, anything. That's `local-exec`. It runs on the machine where Terraform runs, not on the new server.

The reference summary (HashiCorp's three built-in provisioners):

- `local-exec`: invokes a process or script **locally on the machine running the Terraform CLI** (your laptop or a CI/CD build agent).

- `remote-exec`: connects to a newly created remote resource via **SSH or WinRM** to run shell commands **directly inside** that system.
- `file`: copies files or directories securely from the host running Terraform **to the newly spawned remote resource**.

---

2. Basic syntax: `connection` + provisioners

Always remember: a provisioner lives inside a `resource` block. It isn't a standalone top-level block.

`file` and `remote-exec` have to log in to the machine, so they need a `connection` **block**. It holds the type, user, private key and host:

```
resource "aws_instance" "web_server" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  # Connection settings required by remote/file provisioners
  connection {
    type        = "ssh"
    user        = "ubuntu"
    private_key = file("~/.ssh/id_rsa")
    host        = self.public_ip
  }

  # 1. Copy a local configuration file to the new instance
  provisioner "file" {
    source      = "conf/nginx.conf"
    destination = "/tmp/nginx.conf"
  }

  # 2. Run commands inside the instance to install Nginx
  provisioner "remote-exec" {
    inline = [
      "sudo apt-get update",
      "sudo apt-get install -y nginx",
      "sudo mv /tmp/nginx.conf /etc/nginx/nginx.conf",
      "sudo systemctl restart nginx"
    ]
  }
}
```

And `local-exec`, which needs **no** connection because it runs locally:

```
resource "aws_instance" "web" {
  # ...
  provisioner "local-exec" {
    command = "echo The server's IP address is ${self.private_ip}"
  }
}
```

`self` means "this resource". Inside a provisioner or connection block you can't write `aws_instance.web.public_ip` (a resource can't reference itself by name), so you use `self.public_ip`, `self.private_ip`, and so on.

`connection` argument	Meaning
`type`	`"ssh"` (Linux) or `"winrm"` (Windows)
`user`	Login user. Depends on the AMI: `ubuntu` for Ubuntu, `ec2-user` for Amazon Linux/RHEL
`private_key`	The private key's **contents**, so use the `file()` function: `file("/path/to/key")`
`host`	Where to connect: `self.public_ip` for a public instance

---

3. Lab: key pair + security group + EC2 with all three provisioners

Working directory: `root@cka-w2:~/project1` (the same box as Chapter 03; `terraform.log` from the logging lab is still there).

Step 1: generate an SSH key pair locally

```
ssh-keygen -f ram
# Generating public/private ed25519 key pair.
# Enter passphrase (empty for no passphrase):
# Your identification has been saved in ram
# Your public key has been saved in ram.pub

ls
# ram  ram.pub
```

- `ram` = **private key**. It stays on L1 and Terraform uses it in `connection { private_key }`.
- `ram.pub` = **public key**. It gets uploaded to AWS as a key pair and installed on the EC2.
- Newer OpenSSH defaults to **ed25519**, which AWS supports for Linux instances.

Step 2: create the file we'll copy with the `file` provisioner

```
echo "I LOVE Terraform" > /tmp/redhat.txt
```

Step 3: `key.tf`, which uploads the public key to AWS

```
resource "aws_key_pair" "my_blog_key" {
  key_name   = "my_tf_key"
  public_key = file("/root/project1/ram.pub")
}
```

Step 4: `sg.tf`, a security group that allows SSH

```
resource "aws_security_group" "cw_sg_ssh" {
  name = "cw-blog-3-sg-using-terraform"

  #Incoming traffic
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Step 5: init and apply the first two resources

```
ls
# key.tf  ram  ram.pub  sg.tf  terraform.log

terraform init
# - Reusing previous version of hashicorp/aws from the dependency lock file
# - Using previously-installed hashicorp/aws v6.66.0
# Terraform has been successfully initialized!

terraform apply -auto-approve
# + resource "aws_key_pair" "my_blog_key" { ... will be created
# ...

terraform state list
# aws_key_pair.my_blog_key
# aws_security_group.cw_sg_ssh
```

"Created, but I can't see it on the console?" After the apply I opened **EC2 → Instances** (Asia Pacific (Mumbai)) and it said *No instances*. That's expected: so far we'd created only a **key pair**

and a **security group**, with no instance yet. They are listed under **EC2 → Network & Security → Key Pairs / Security Groups**, not under Instances. (Also check that the console region matches the provider's `region`. That's the other common reason for "missing" resources.)

Step 6: the EC2 instance with `connection` + `file` + `remote-exec` + `local-exec`

The target config (Amazon Linux, so `ec2-user` and `dnf`):

```
resource "aws_instance" "web_server" {
  ami             = "ami-066c4849e6b3a1e3d"
  instance_type   = "t2.nano"
  key_name        = "my_tf_key"
  security_groups = ["cw-blog-3-sg-using-terraform"]

  # Connection settings required by remote/file provisioners
  connection {
    type        = "ssh"
    user        = "ec2-user"
    private_key = file("/root/project1/ram")
    host        = self.public_ip # refers to the public IP of this particular instance
  }

  # 1. Copy a local file to the new instance  (= scp)
  provisioner "file" {
    source      = "/tmp/redhat.txt"
    destination = "/tmp/redhat1.txt"
  }

  # 2. Run commands inside the instance  (= ssh + run)
  provisioner "remote-exec" {
    inline = [
      "sudo dnf update",
      "sudo dnf install -y httpd",
      "sudo echo hi this my basic web server > /var/www/html/index.html",
      "sudo systemctl enable --now httpd"
    ]
  }

  # 3. Run a command on the local machine (L1)
  provisioner "local-exec" {
    command = "echo the servers IP address is ${self.private_ip} > /root/project1/my-node-ip
  }
}
```

Mistakes I made in the first draft, and their fixes. The first version typed in `vi` had three bugs worth remembering:

First draft	Problem	Fix
`resource "aws-instance" "web_server"`	Resource **type** uses a hyphen. AWS types are `aws_instance` with an underscore	`resource "aws_instance" ...`
`}` closed the resource **before** `connection {`	`connection` and the provisioners ended up **outside** the resource. They must be nested inside it	Move the closing `}` to the very end
`private_key = "file("/root/project1/ram")"`	Wrapped in quotes, it's a **literal string**, not a function call	`private_key = file("/root/project1/ram")`

Step 7: Terraform crashed, so the provisioners moved to a `null_resource`

Running the all-in-one version above ended in:

```
!!!!!!!!!!!!!!!!!!!!!!!! TERRAFORM CRASH !!!!!!!!!!!!!!!!!!!!!!!!
Terraform crashed! This is always indicative of a bug within Terraform.
...
panic: value is unknown
...
github.com/hashicorp/terraform/internal/terraform.(*Graph).walk.func1.1()
```

`value is unknown` means Terraform tried to use a value that is **only known after apply** (like the new instance's IP) at a point where it wasn't available yet. A real error would give a proper message. A panic is a Terraform bug, but the practical fix was to restructure.

The re-run separated **creating the server** from **configuring the server**. The EC2 instance is created first. A separate `null_resource` (from the `hashicorp/null` provider, a resource that creates nothing in the cloud) holds the connection and provisioners and points at the finished instance. Pattern:

```
resource "aws_instance" "web_server" {
  ami             = "ami-066c4849e6b3a1e3d"
  instance_type   = "t2.nano"
  key_name        = "my_tf_key"
  security_groups = ["cw-blog-3-sg-using-terraform"]
}

resource "null_resource" "configure_server" {
  connection {
    type        = "ssh"
    user        = "ec2-user"
    private_key = file("/root/project1/ram")
    host        = aws_instance.web_server.public_ip   # not self — self is the null_resource
  }

  provisioner "file"        { ... }
  provisioner "remote-exec" { ... }
  provisioner "local-exec" {
    command = "echo the servers IP address is ${aws_instance.web_server.private_ip} > /root/
  }
}
```

- The reference `aws_instance.web_server.public_ip` also creates an **implicit dependency** (Chapter 02), so the null_resource always waits for the instance.
- `null_resource` needs the `hashicorp/null` provider, so **re-run** `terraform init` after adding it.
- On Terraform ≥ 1.4 the built-in `terraform_data` resource does the same job with no extra provider: `resource "terraform_data" "configure_server" { ... }`.

Step 8: the apply that worked

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_instance.web_server: Creating...
aws_instance.web_server: Creation complete after 15s [id=i-08009fcf02e77e2b4]
null_resource.configure_server: Creating...
null_resource.configure_server: Provisioning with 'file'...
null_resource.configure_server: Provisioning with 'remote-exec'...
null_resource.configure_server (remote-exec): Connecting to remote host via SSH...
null_resource.configure_server (remote-exec):   Host: 13.203.105.46
null_resource.configure_server (remote-exec):   User: ec2-user
null_resource.configure_server (remote-exec):   Password: false
null_resource.configure_server (remote-exec):   Private key: true
null_resource.configure_server (remote-exec):   Certificate: false
null_resource.configure_server (remote-exec):   SSH Agent: false
null_resource.configure_server (remote-exec):   Checking Host Key: false
null_resource.configure_server (remote-exec):   Target Platform: unix
null_resource.configure_server (remote-exec): Connected!
null_resource.configure_server (remote-exec): Waiting for process with pid 2515 to finish.
null_resource.configure_server: Still creating... [00m10s elapsed]
...
null_resource.configure_server: Still creating... [01m20s elapsed]
```

Reading the output:

- `2 to add` = `aws_instance` + `null_resource`. The key pair and SG already existed from Step 5.
- The order is exactly what the dependency implies: **instance first, then provisioners in the order they are written** (`file` → `remote-exec` → `local-exec`).
- `Private key: true`, `Password: false` → it logged in with the `ram` key via the `connection` block. `Connected!` proves the key pair, SG port 22 and user `ec2-user` were all correct.
- `Waiting for process with pid 2515 to finish.` isn't a Terraform message. It's `dnf` inside the instance saying another dnf process holds the package lock. On a freshly booted Amazon Linux, cloud-init / `dnf makecache` is usually still running, so `remote-exec` waits. That's why "Still creating..." kept ticking. The screenshots stop at 1m20s, so the end of this run isn't captured.

---

4. Things to watch out for (from reviewing this lab)

1. `sudo dnf update` **has no** `-y`**.** In a non-interactive SSH session, dnf's "Is this ok [y/N]" prompt gets no answer and the command aborts. Use `sudo dnf update -y` (or `-y` on every command in `inline`).

2. `sudo echo ... > /var/www/html/index.html` **fails with *Permission denied*.** `sudo` applies to `echo`, but the `>` redirect is done by *your* shell as `ec2-user`. Use `echo "hi this my basic web server" | sudo tee /var/www/html/index.html`.
3. **The security group has no `egress` block.** When Terraform manages an `aws_security_group`, it **removes AWS's default allow-all outbound rule**. With no egress the instance can't reach the dnf repos, so updates/installs hang. SSH still works, because security groups are stateful and replies to inbound traffic are allowed. Add:

   ```
   egress {
     from_port   = 0
     to_port     = 0
     protocol    = "-1"
     cidr_blocks = ["0.0.0.0/0"]
   }
   ```

   and open **port 80** too if you want to open the httpd page in a browser.
4. **Key name mismatch.** `key.tf` names the pair `my_tf_key` (underscore), but the instance drafts say `key_name = "my_tf-key"` (hyphen). AWS would reject that with `InvalidKeyPair.NotFound`. Better still, **don't hard-code names. Reference the resources**: `key_name = aws_key_pair.my_blog_key.key_name` and `vpc_security_group_ids = [aws_security_group.cw_sg_ssh.id]`. That removes typos and gives Terraform the dependency order for free.
5. **Provisioners only run at creation time.** If you edit the `inline` commands later, the next `apply` does **not** re-run them on the existing instance. You need `terraform apply -replace=...` (Chapter 03) or, with a null_resource, a `triggers` map that changes.
6. **A failed provisioner taints the resource.** If `remote-exec` fails during create, the resource is marked **tainted** (Chapter 03) and gets destroyed and recreated on the next apply. Change this with `on_failure = continue` (default is `fail`).
7. **Destroy-time provisioners exist:** `when = destroy` runs a provisioner just before the resource is destroyed (e.g. deregister from a load balancer or backup a file).

Why HashiCorp calls provisioners a "last resort"

Terraform can't plan what a script will do. It isn't in the state, it isn't shown in `plan`, and it isn't idempotent. Prefer, in order: `user_data` **/ cloud-init** (Chapter 03, runs at first boot), a **pre-baked AMI** (Packer), or a **config-management tool** (Ansible) run after Terraform. Provisioners are for gaps these can't cover. `local-exec` for small glue steps (write an inventory file, call an API, trigger Ansible) is the most common legitimate use.

Command reference (this chapter)

```
ssh-keygen -f ram                        # private key "ram" + public key "ram.pub"
echo "I LOVE Terraform" > /tmp/redhat.txt
terraform init                           # re-run after adding null_resource (hashicorp/null
terraform apply -auto-approve
terraform state list
cat /root/project1/my-node-ip.txt        # output of the local-exec provisioner
ssh -i ram ec2-user@<public-ip>          # verify by hand: cat /tmp/redhat1.txt, systemctl s
terraform destroy -auto-approve          # clean up the lab
```

Still to come

- **Terraform plugin cache**: a local store (`plugin_cache_dir` / `TF_PLUGIN_CACHE_DIR`) that saves downloaded providers such as `hashicorp/aws` and `hashicorp/null` , so each `terraform init` doesn't download them again.